CHAPTER 6 — SOFTWARE METRICS

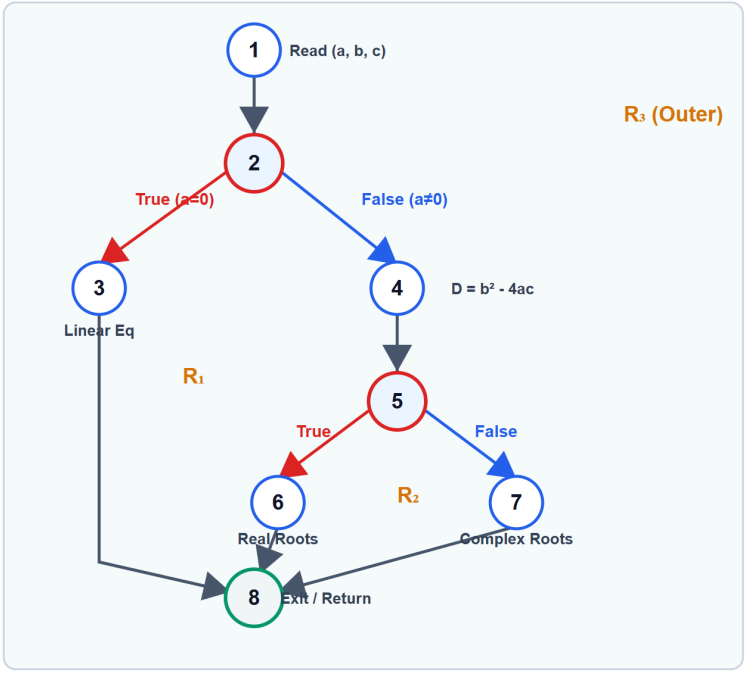
Structural Diagrams & Tracing Pack

Section 1: Control Flow Graph (CFG) & Cyclomatic Complexity Models

Q6.DGM01. Control Flow Graph Construction and Cyclomatic Complexity Derivation for Quadratic Equation Classifier

Figure 1: Control Flow Graph (CFG) for Quadratic Equation Classifier

McCabe Cyclomatic Complexity V(G) = E - N + 2P = 9 - 8 + 2(1) = 3



Topological Metric Summary

- Number of Nodes (N) = 8
- Number of Directed Edges (E) = 9
- Predicate Decision Nodes (P_pred) = 2 (Nodes 2 and 5)
- Planar Regions (R) = 2 enclosed + 1 outer = 3

Method 1: V(G) = E - N + 2P = 9 - 8 + 2(1) = 3
Method 2: V(G) = P_pred + 1 = 2 + 1 = 3
Method 3: V(G) = Regions = 3

Linearly Independent Basis Paths (Basis = 3)

Path 1 (Linear Equation Branch):
1 → 2 → 3 → 8

Path 2 (Two Real Roots Branch):
1 → 2 → 4 → 5 → 6 → 8

Path 3 (Complex Roots Branch):
1 → 2 → 4 → 5 → 7 → 8

✓ Every edge and node is traversed by at least one basis path.

Figure 1: Control Flow Graph (CFG) for Quadratic Equation Classifier

Topological Graph Analysis & Cyclomatic Complexity:

1. **Topological Graph Specification (G):**
2. **Node Set ($N = 8$):** $\{1, 2, 3, 4, 5, 6, 7, 8\}$
3. **Directed Edge Set ($E = 9$):** Directed transfers of control between sequential nodes and branching paths.
4. **Predicate Decision Nodes ($P_{pred} = 2$):** Node 2 (if `a == 0`) and Node 5 (if `D > 0`), each having out-degree 2.
5. **Connected Components (P):** $P = 1$ (single coherent program flow).
6. **Cyclomatic Complexity Derivation ($V(G)$):**
7. **Method 1 (Edge-Node Formula):**

$$V(G) = E - N + 2P = 9 - 8 + 2(1) = 3$$

8. **Method 2 (Predicate Decision Nodes Formula):**

$$V(G) = P_{pred} + 1 = 2 + 1 = 3$$

9. **Method 3 (Planar Regions Formula):**

$$V(G) = \text{Enclosed Regions} + \text{Outer Region} = 2 + 1 = 3$$

10. **Linearly Independent Basis Paths (Basis = 3):**
11. **Path 1:** $1 \to 2 \to 3 \to 8$ (Linear equation branch)
12. **Path 2:** $1 \to 2 \to 4 \to 5 \to 6 \to 8$ (Two real roots branch)
13. **Path 3:** $1 \to 2 \to 4 \to 5 \to 7 \to 8$ (Complex / equal roots branch)

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 422–425; companion slides Ch.8.

ملاحظة أكاديمية هامة (Academic Note)

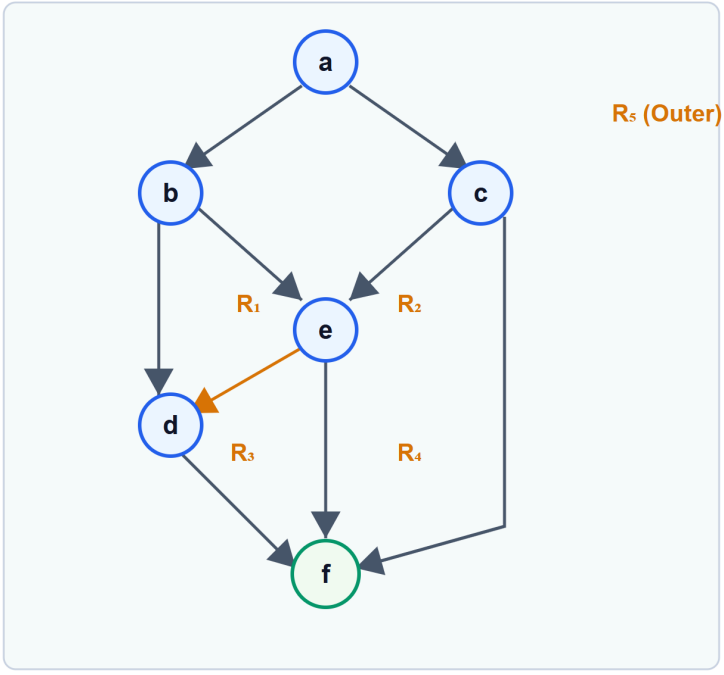
ملاحظة الطالب والفخ الامتحاني:

- تتطابق الطرق الثلاث لحساب التعقيد الإعصاري:
- $E - N + 2 = 9 - 8 + 2 = 3$.
- عقد القرار + 1 = $2 + 1 = 3$.
- عدد المناطق المحصورة + المنطقة الخارجية = $2 + 1 = 3$.
- يلزم اختيار 3 مسارات أساسية مستقلة وفق $V(G) = 3$؛ هذا لا يضمن وحده تغطية كل شرط أو كل حافة.

Q6.DGM02. Aggarwal & Singh's Canonical Flow Graph (Figure 8.21 / Slide 133) Analysis

Figure 8.21: Canonical Flow Graph & Cyclomatic Complexity Verification

K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 423 / Companion Slide 133



Topological Invariants & Equations

- Node Set: {a, b, c, d, e, f} → N = 6
- Directed Edge Set: 9 edges → E = 9
- Predicate Decision Nodes: {a, b, c, e} → P_pred = 4
- Planar Enclosed Regions: 4 internal + 1 external = 5

Method 1: V(G) = E - N + 2P = 9 - 6 + 2(1) = 5
Method 2: V(G) = P_pred + 1 = 4 + 1 = 5
Method 3: V(G) = Total Regions = 5

Aggarwal's Five Independent Paths

Path 1: a → c → f
Path 2: a → c → e → f
Path 3: a → c → e → d → f
Path 4: a → b → d → f
Path 5: a → b → e → f

✓ Basis path set is minimal, linearly independent, and complete.

Figure 2: Aggarwal Canonical Flow Graph (Figure 8.21)

Topological Verification of Canonical Figure 8.21:

1. **Graph Specification:**
2. **Nodes ($N = 6$):** $\{a, b, c, d, e, f\}$, with a as entry and f as exit.
3. **Directed Edges ($E = 9$):**
   $(a \to b), (a \to c), (b \to d), (b \to e), (c \to e), (c \to f), (d \to f), (e \to d), (e \to f)$.
4. **Predicate Decision Nodes:** Four nodes (a, b, c, e) possess out-degree 2.
5. **Connected Components (P):** $P = 1$.
6. **Cyclomatic Complexity Evaluation:**
7. **Method 1:** $V(G) = E - N + 2P = 9 - 6 + 2(1) = 5$
8. **Method 2:** $V(G) = P_{pred} + 1 = 4 + 1 = 5$
9. **Method 3:** Planar regions = 4 internal enclosed regions (R_1, R_2, R_3, R_4) + 1 external region (R_5) = 5.
10. **The Five Linearly Independent Execution Paths:**
11. **Path 1:** $a \to c \to f$
12. **Path 2:** $a \to c \to e \to f$
13. **Path 3:** $a \to c \to e \to d \to f$
14. **Path 4:** $a \to b \to d \to f$
15. **Path 5:** $a \to b \to e \to f$

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 423, Fig. 8.21; companion slide 133.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - هذا الرسم القياسي من كتاب أغروال (Fig. 8.21) يمتلك 6 عقد و9 حواف و4 عقد قرار، ولهذا ينتج تعقيد إعصاري $V(G) = 5$ و5 مسارات مستقلة تماماً.

Section 2: Data Structure Tracing & Lifespan Traces

Q6.DGM03. Variable Cross-Reference List Construction & Data Extraction (Exercise 6.14)

Given Source Code Listing: - Line 1: int `a, b, c,` sum, avg; (Declarations) - Line 2: read(`a`); - Line 3: read(`b`); - Line 4: read(`c`); - Line 5: sum `= a + b + c;` - Line 6: avg `=` sum / 3; - Line 7: print(avg); - Line 8: print(sum);

Structured Variable Cross-Reference Ledger:

- **Variable a:** Declared at Line 1; Defined at Line 2; Referenced at Line 5. Total Occurrences = 3.
- **Variable b:** Declared at Line 1; Defined at Line 3; Referenced at Line 5. Total Occurrences = 3.
- **Variable c:** Declared at Line 1; Defined at Line 4; Referenced at Line 5. Total Occurrences = 3.
- **Variable sum:** Declared at Line 1; Defined at Line 5; Referenced at Lines 6 and 8. Total Occurrences = 4.
- **Variable avg:** Declared at Line 1; Defined at Line 6; Referenced at Line 7. Total Occurrences = 3.

Extracted Data Characteristics: - Total distinct variables ($VARS$ / η_2): 5 (a, b, c, sum, avg). - Total variable token occurrences (N_2): 16 occurrences across the 8 statements.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 273–275; companion slides 41–44. Exercise 6.14.

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - قائمة الإسناد المتبادل (Cross-Reference List) توثق لكل متغير سطر الإعلان وسطر الإسناد (Assignment) وأسطر الاستخدام (References) بدقة لتتبع مسار البيانات داخل الكود.

Q6.DGM04. Trace live variables, average variable life (γ), variable spans (SP), and module weakness (WM) in the six-statement example.

Figure 3: Live Variables & Module Weakness (WM)

K.K. Aggarwal & Yogesh Singh (3rd ed.), Companion Slides 54 & 68 / Scan pp. 278–280



Aggarwal's Exact Live Variable Metrics

- Executable Statements Count (n) = 6
- Total Unique Variables = 4 (x, y, z, p)
- Sum of Live Variable Counts: 1+2+2+2+1+0 = 8

Average Live Variables (LV̄):

LV̄ = (Sum of LV) / n = 8 / 6 ≈ 1.33 variables

Average Life of Variables (γ):

γ = (Sum of LV) / (Total Vars) = 8 / 4 = 2.0

Module Weakness (WM) Governing Law

Source Formulation (Slide 54 & 68):

WM = LV̄ × γ

Substitution for this program:

WM = 1.333 × 2.0 = 2.67

Canonical Slide 68 Verification:

Sum LV = 174, n = 53, Total Vars = 11

LV̄ = 3.28, γ = 15.8 → WM = 3.28 × 15.8 = 51.8

Figure 3: Live Variable Lifespans & Module Weakness

Authored practice trace using the course definitions (the six-statement data below are a constructed example, not a transcription of a textbook exercise).

Conventions: Count live variables immediately **after** each statement. A variable is live at that point when it has been defined and will be used later before being redefined. For spans, count only statements between successive uses; a definition is not a use.

1. **Trace:**
2. S_1: x = read(); $\{x\}$, so $LV(S_1) = 1$.
3. S_2: y = read(); $\{x, y\}$, so $LV(S_2) = 2$.
4. S_3: z = x + y; y has its last use; $\{x, z\}$, so $LV(S_3) = 2$.
5. S_4: p = z * 2; z has its last use; $\{x, p\}$, so $LV(S_4) = 2$.
6. S_5: print(x); x has its last use; $\{p\}$, so $LV(S_5) = 1$.
7. S_6: print(p); p has its last use; $\emptyset$, so $LV(S_6) = 0$.
8. **Live-variable metrics:** There are $n = 6$ executable statements and four distinct variables. Thus $\sum LV = 1 + 2 + 2 + 2 + 1 + 0 = 8$, $\overline{LV} = 8/6 \approx 1.33$ variables per statement, and $\gamma = 8/4 = 2.0$ statement-count units per variable under this convention.
9. **Module weakness:**

$$WM = \overline{LV} \times \gamma = (8/6) \times 2 = 8/3 \approx 2.67.$$

10. **Variable span:** Between successive uses, $SP = b - a - 1$. x is used at S_3 and S_5, so $SP(x) = 5 - 3 - 1 = 1$. The other variables have only one use each and add no span. Therefore $SP_{avg} = 1/1 = 1$ statement for the single observed span.

Source for definitions and formulas: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 275–280; companion slides 45–68. Numerical trace is authored practice.

ملاحظة أكاديمية هامة (Academic Note)

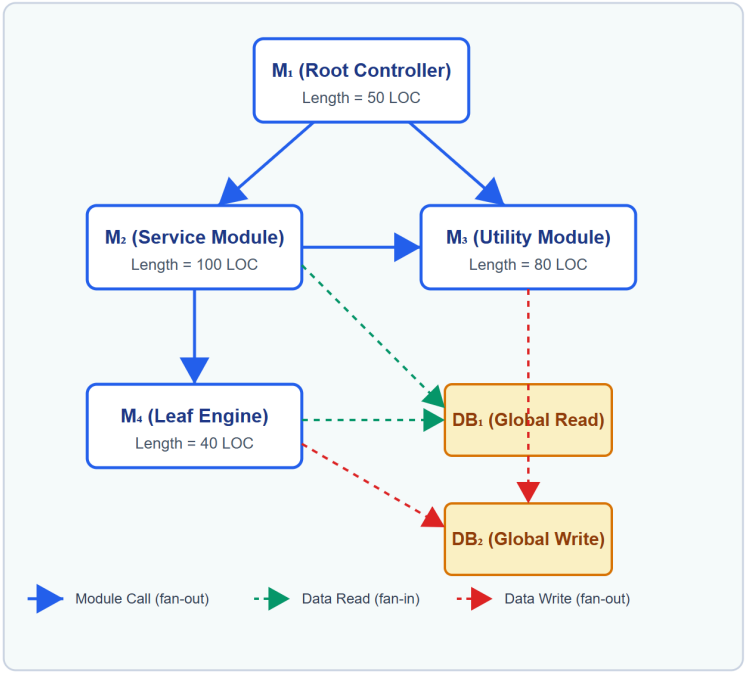
ملاحظة الطالب والفخ الامتحاني:

- المثال هنا مصنوع للتدريب، مو نص تمرين من الكتاب. الرسم والحل يستخدمان اصطلاح العدّ بعد تنفيذ التعليمة، لذلك صارت القيم نفسها في الاثنين.

Q6.DGM05. Multi-Tier Information Flow Call Graph & Henry–Kafura Metric (HKM)

Figure 4: Information Flow Architecture & Henry–Kafura Metric (HKM)

Pressman 5e §19.4.2 (p. 524) vs. Aggarwal Slide 74 Boundary Conventions



Pressman 5e §19.4.2 Model (HKM)

HKM = length × [fin + fout]²

- fin = calling modules + data structures read
- fout = called modules + data structures updated

Module Results:

HKM(M₁) = 50 × [0 + 2]² = 50 × 4 = 200
HKM(M₂) = 100 × [2 + 2]² = 100 × 16 = 1,600 (Bottleneck)
HKM(M₃) = 80 × [2 + 1]² = 80 × 9 = 720
HKM(M₄) = 40 × [2 + 1]² = 40 × 9 = 360

Aggarwal Slide 74 Boundary Procedure

- Root Component: If raw fin = 0 → assign fin = 1
- Leaf Component: If raw fout = 0 → assign fout = 1

Slide 74 Adjusted Value for Root M₁:

Because raw fin = 0, adjusted fin = 1.
HKM_slide(M₁) = 50 × [1 + 2]² = 50 × 9 = 450

- Note: Slide 74 explicitly cautions that basic fan-in models may penalize highly reused components with large scores.

Figure 4: Information Flow Architecture & Henry-Kafura Metric

Architectural Topology & Metric Comparison:

1. **Modular Parameters (Pressman 5e §19.4.2 Model):**
2. **Root M_1 (50 LOC):** $fin = 0$ callers, 0 reads = 0; $fout = 2$ calls (M_2, M_3), 0 writes = 2.

$$HKM(M_1) = 50 \times [0 + 2]^2 = 50 \times 4 = 200$$

3. **Service M_2 (100 LOC):** $fin = 1$ caller (M_1) + 1 read (DB_1) = 2; $fout = 2$ calls (M_3, M_4) + 0 writes = 2.

$$HKM(M_2) = 100 \times [2 + 2]^2 = 100 \times 16 = 1,600 \text{ (largest calculated value in this example)}$$

4. **Utility M_3 (80 LOC):** $fin = 2$ callers (M_1, M_2) + 0 reads = 2; $fout = 0$ calls + 1 write (DB_2) = 1.

$$HKM(M_3) = 80 \times [2 + 1]^2 = 80 \times 9 = 720$$

5. **Leaf M_4 (40 LOC):** $fin = 1$ caller (M_2) + 1 read (DB_1) = 2; $fout = 0$ calls + 1 write (DB_2) = 1.

$$HKM(M_4) = 40 \times [2 + 1]^2 = 40 \times 9 = 360$$

6. **Course Slide 74 Boundary Procedure Comparison:**
7. For root component M_1, because raw $fin = 0$, slide convention sets $fin = 1$:

$$HKM_{slide}(M_1) = 50 \times [1 + 2]^2 = 50 \times 9 = 450$$

8. Slide 74 explicitly notes that basic fan-in models can penalize reused utility components.

Source: Pressman, R., 5th ed., printed p. 524; K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 69–77.

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- انتبه لتطبيق اصطلاح الشريحة 74: إذا كان $fan_in = 0$ للوحدة الجذرية العليا يعوض بـ 1 لمنع تصفير القيمة.
- يشمل fan_in الدوال المستدعية + هياكل البيانات المقروءة.
- يشمل fan_out الدوال المستدعاة + هياكل البيانات المحدثة أو المكتوبة.

Q6.DGM06. Goal–Question–Metric (GQM) Hierarchical Architectural Tree

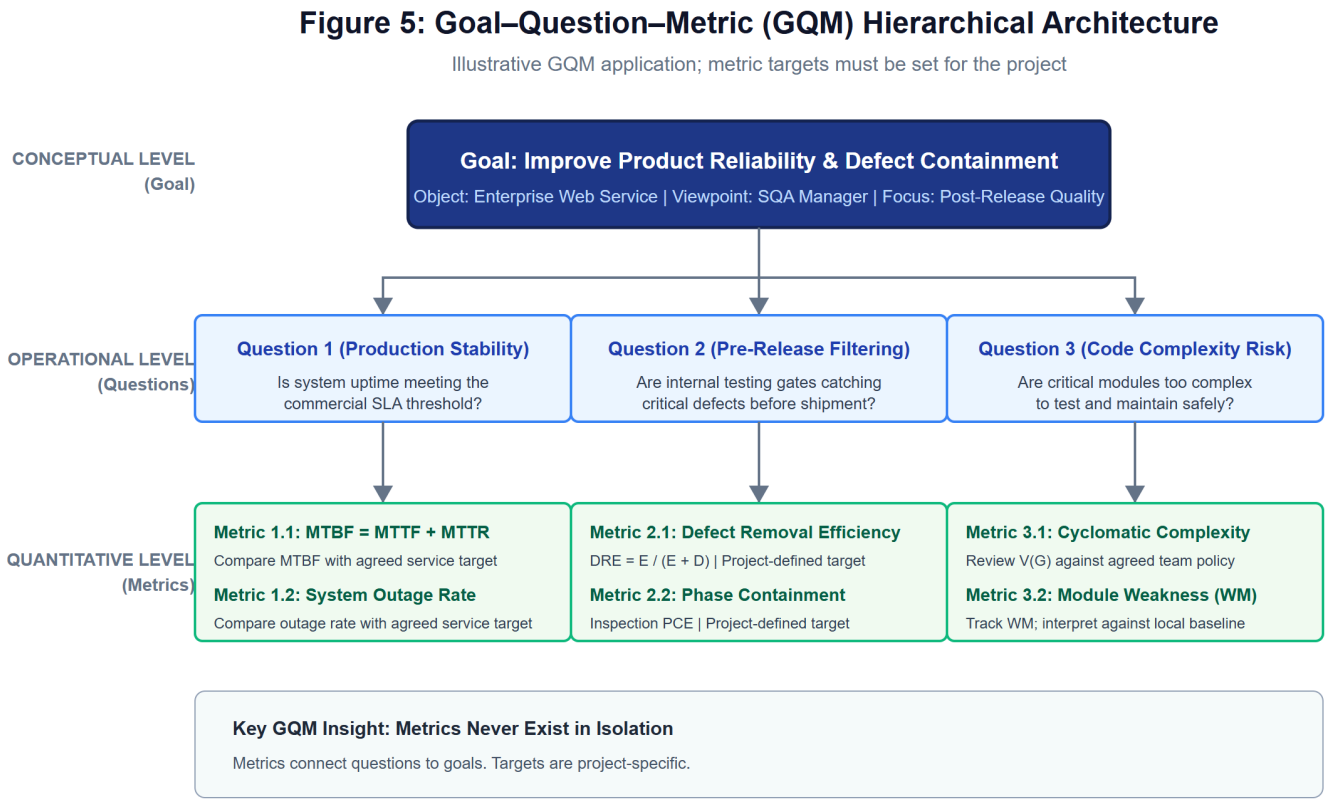


Figure 5: Goal-Question-Metric (GQM) Hierarchical Architecture

Three-Tier GQM Hierarchical Mapping:

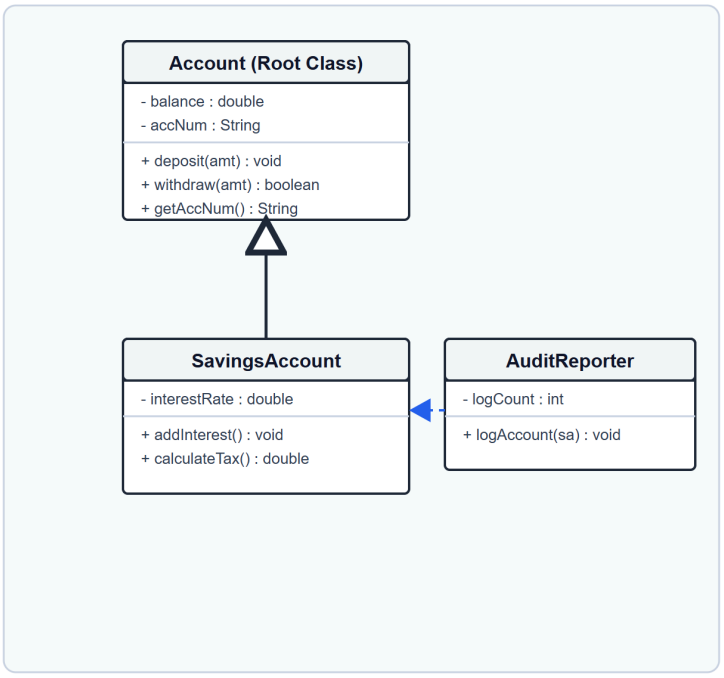
1. **Conceptual Level (Goal):**
2. *Goal:* Improve software operational reliability and post-release defect containment.
3. *Object:* Enterprise Web Portal; *Viewpoint:* SQA Manager; *Context:* Production Release.
4. **Operational Level (Questions):**
5. *Question 1 (Production Stability):* Is system uptime meeting contractual SLAs?
6. *Question 2 (Pre-Release Filtering):* Are internal QA gates successfully catching defects before shipment?
7. *Question 3 (Structural Complexity Risk):* Are core modules too complex to test and maintain safely?
8. **Quantitative Level (Metrics):**
9. *For Q1:* Track $MTBF = MTTF + MTTR$ and unplanned downtime; compare both with service targets defined for this project.
10. *For Q2:* Track $DRE = \frac{E}{E+D}$ and phase-containment effectiveness; compare them with targets chosen from local baselines.
11. *For Q3:* Track McCabe Cyclomatic Complexity $V(G)$ against project policy and Module Weakness $WM = \overline{LV} \times \gamma$ against a local baseline.

Targets and metric choices in this authored example are illustrative; no numeric threshold is asserted as universal. *Source: Leach, R., printed pp. 39–40 (GQM hierarchy).*

Q6.DGM07. Object-Oriented Architecture & Chidamber & Kemerer (CK) Metric Suite

Figure 6: Object-Oriented Architecture & Chidamber & Kemerer (CK) Metric Suite

K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 288–294 / Slides 80–85



CK Suite Metrics for Class `Account`

- WMC = 3 (deposit, withdraw, getAccNum)
- DIT = 0 (Root class with no ancestors)
- NOC = 1 (Immediate subclass: SavingsAccount)
- CBO = 0 (Calls no external classes)
- RFC = 3 (Internal methods only; invokes 0 external)

LCOM Derivation for `Account`:

Pairs sharing balance: (deposit, withdraw) → Q = 1

Disjoint pairs: (dep, getAccNum), (wdr, getAccNum) → P = 2

LCOM = P - Q = 2 - 1 = 1

Metrics for Subclass & Client

Class `SavingsAccount`:

- DIT = 1 (One level below Account)
- NOC = 0 (Leaf class)
- RFC = 4 (2 local methods + 2 invoked from Account)

Class `AuditReporter`:

- CBO = 1 (Coupled to SavingsAccount)
- DIT = 0 (Standalone root class)

Figure 6: Object-Oriented Architecture & CK Metric Suite

Structural Analysis of 3-Class Design:

1. **Class Account (Root Classifier):**
2. Attributes: balance, accNum. Methods: deposit(), withdraw(), getAccNum().
3. **WMC = 3** (unit method complexities).
4. **DIT = 0** (root class, maximum path to root is 0).
5. **NOC = 1** (direct child is SavingsAccount).
6. **CBO = 0** (no external class dependencies).
7. **RFC = 3** (local methods only, invokes 0 external methods).
8. **LCOM = 1:** Methods pair $(deposit, withdraw)$ shares balance ($Q = 1$); pairs $(deposit, getAccNum)$ and $(withdraw, getAccNum)$ share no variables ($P = 2$). Thus, $LCOM = P - Q = 2 - 1 = 1$.
9. **Class SavingsAccount (Subclass of Account):**
10. **DIT = 1** (one step from root Account).
11. **NOC = 0** (leaf class).
12. **RFC = 4** (2 local methods + 2 invoked from Account).
13. **Class AuditReporter (Client Collaborator):**
14. **CBO = 1** (coupled to SavingsAccount).
15. **DIT = 0** (standalone root class).

> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 288–294; companion slides 80–85.*

ملاحظة أكاديمية هامة (Academic Note)

ملاحظة الطالب والفخ الامتحاني:

- حزمة CK الستة:
- WMC: مجموع أوزان دوال الكلاس.
- DIT: أطول مسار إلى الجذر (Root).
- NOC: عدد الأبناء المباشرين.
- CBO: عدد الكلاسات الأخرى المرتبطة بهذا الكلاس.
- RFC: عدد دوال الكلاس + الدوال الخارجية التي تستدعيها.
- LCOM: مقياس غياب التماسك ($P - Q$).

Trace the three basis paths of Figure 1 and calculate $V(G)$ using all 3 methods.

Trace the five paths shown in Aggarwal's Figure 8.21 and verify the corresponding graph paths without looking at the worked explanation.

Re-derive the live variable set and lifespan γ for statements S_1 to S_6.

Explain the difference between Slide 74 boundary rules and Pressman's HKM for root components.

Calculate WMC, DIT, NOC, CBO, RFC, and LCOM for the Account class in Figure 6.